

DynamoLLM: 设计 LLM 推理集群以提高性能和能源效率

Jovan Stojkovic、张超杰[†]、Íñigo Goiri[†]、Josep Torrelas、Esha Choukse[†]
University of Illinois at Urbana-Champaign[†]Microsoft Azure Research - Systems

Abstract—生成式大语言模型 (LLM) 的快速发展和广泛采用使其成为各种应用程序中的关键工作负载。如今，LLM 推理集群收到大量具有严格服务级别目标 (SLO) 的查询。为了达到所需的性能，这些模型在耗电的 GPU 上执行，导致推理集群消耗大量能源，从而导致过多的碳排放。幸运的是，我们发现有很好的机会利用推理计算属性中的 *heterogeneity* 和推理工作负载中的 *fluctuations* 来显著提高能源效率。然而，这种多样化和动态的环境创建了一个巨大的搜索空间，其中不同的系统配置 (*e.g.*、实例数量、模型并行性和 GPU 频率) 会转化为不同的能源性能权衡。为了应对这些挑战，我们提出了 **DynamoLLM**，这是第一个适用于 LLM 推理环境的能源管理框架。DynamoLLM 自动动态地重新配置推理集群，以优化服务性能 SLO 下的 LLM 服务的能源和成本。我们表明，在服务级别，DynamoLLM 节省了 53% 的能源和 38% 的运营碳排放，并为客户降低了 61% 的成本，同时满足延迟 SLO。

阳离子 [7]、[17]、[21]、[31]、[61]、[80]。具体来说，我们观察到 LLM 推理计算属性中的 *heterogeneity* 和 LLM 推理工作负载中的 *fluctuations* 创建了一个具有较大变化的动态环境。此类差异源自：(1) 具有不同输入/输出令牌长度的请求，(2) 不同 LLM 的不同计算属性，以及 (3) 使用 LLM 的服务所需的不同 SLO。

具有大量输入令牌的请求属于计算密集型，因此对 GPU 频率敏感。相反，具有少量输入令牌和大量输出令牌的请求计算量较低，但内存需求较高。降低 GPU 频率可以节省能源，而不会显着影响性能。此外，模型参数的数量也会影响 LLM 对 GPU 数量和 GPU 频率的敏感度。最后，根据当前使用 LLM 的服务，SLO 要求可以严格要求高性能配置，也可以宽松允许较低性能但更节能的配置。重要的是，这些特征由于负载波动和请求的动态分布而迅速变化。这种动态变化会导致在给定点上节能的系统配置很快变得次优。这需要一个 *dynamic approach* 来进行资源管理。

一、简介

生成式大语言模型 (LLM) 的采用呈指数级增长，使其成为众多技术进步和应用的核心。今天，我们看到法学硕士在各个领域的用例，例如医疗保健[52]、开发人员生产力[13]、数据分析[68]、教育[5]等。随着大语言模型中越来越受欢迎，推理集群每天会收到数百万次查询[27]，从而形成具有复杂软件和昂贵硬件系统的大型基础设施。

为了满足这些不断增长的计算需求，研究人员提出了各种软件[9]、[26]、[35]、[73]、[81]和硬件[4]、[50]、[78]技术。此类技术提高了 LLM 推理集群的性能效率。然而，一个被很大程度上忽视的方面是这些环境的能源消耗[58]、[60]。在耗电的 GPU 上运行的大语言模型量能源需求已成为一个重大问题。随着这些模型成为各种服务不可或缺的一部分，在保持高性能的同时最大限度地减少能源消耗和碳排放至关重要。

为了解决这一差距，本文首先描述了 LLM 推理工作负载的能效特性。我们的特征强调，此类环境提出了一系列独特的挑战，与为传统数据中心应用量身定制的现有能源管理方案不同。

为了为实现节能和可持续的 LLM 推理集群铺平道路，本文引入了 **DynamoLLM**，这是第一个用于 LLM 推理环境的能源管理框架。DynamoLLM 利用 LLM 推理工作负载的独特属性来降低能耗，同时满足性能 SLO。系统使用模型及其工作负载的能源性能配置文件来 *automatically* 和 *dynamically* 选择节能配置。它利用多个旋钮，包括缩放服务器实例的数量、跨 GPU 的模型并行性以及 GPU 频率缩放。

为了处理工作负载异构性，DynamoLLM 维护不同配置的 LLM 实例池，这些实例最适合不同类型的传入请求。例如，与具有许多输入和输出令牌的请求相比，处理和输出较少令牌的请求在跨较少频率运行的较少 GPU 并行的模型上运行效率更高。由于请求分布随时间变化，DynamoLLM 会动态调整池的大小。随着时间的推移，这些池可以合并为更少的池或分为多个池，从而在资源的合理规模和碎片化之间提供平衡。为了有效地管理

DynamoLLM 使用控制器层次结构来降低计算复杂性并消除集中式瓶颈。每一级的控制器在上一级施加的条件下运行，计算其专用旋钮，并将进一步的约束转发给下一级的控制器。最后，为了实现不同配置之间的频繁、平滑过渡，DynamoLLM 包含最小化或隐藏重新配置开销的技术。因此，系统可以在不断变化的工作负载需求下保持高水平的效率和服务质量。

对具有运行来自主要云提供商的生产级跟踪的大型 GPU 集群的 DynamoLLM 进行的评估表明，DynamoLLM 非常有效：它节省了 53% 的能源，减少了 38% 的运营碳排放，并为客户降低了 61% 的成本，同时满足延迟 SLO。

本文的贡献如下：

- 对节能大语言模型机会的分析，根源于推理工作负载的异质性和波动。
- DynamoLLM 的设计和实现，这是一个用于 LLM 推理的高性能和能源优化框架。
- 使用生产级跟踪在大型平台上评估 DynamoLLM。

二.背景

LLM 的计算阶段生成式 LLM [34]、[38]、[56]、[67]、[80] 是自回归的：它们并行处理整个输入，并串行生成输出标记。这个属性导致两个计算上不同的阶段[49]、[50]。首先是预填充阶段，其中并行计算输入标记。这是一个计算密集型阶段，并随着输入令牌的数量而扩展。第二个是解码阶段，其中每个输出令牌是根据迄今为止看到的所有令牌串行生成的。这是一个内存密集型阶段，并随着输出令牌的数量而扩展。

LLM 的性能指标 为了评估性能，我们使用：第一个令牌的时间 (TTFT)、令牌之间的时间 (TBT) 和吞吐量 [50]、[63]。TTFT 是生成第一个输出令牌的延迟；而 TBT 是生成每个新输出令牌的延迟。为了量化能源效率，我们在满足特定延迟 SLO 的情况下以瓦时 (Wh) 为单位测量能源消耗。SLO 根据不同任务的用例而有所不同。对于延迟敏感的任务，TTFT 和 TBT 都是具有严格 SLO 的重要指标。我们根据可实现的最大性能定义 TTFT 和 TBT 的 SLO，如表 IV 中所述。

LLM 并行性 单个模型可以跨 GPU 划分，以提高性能并允许更大的内存占用。LLM 推理通常使用管道和张量并行性。管道并行性 (PP) 在 GPU 之间划分 LLM 层，同时将层的所有运算符/张量保留在 GPU 上。然后 GPU 仅在两个连续阶段之间进行通信。张量并行 (TP) 将每层的一个切片分配给每个 GPU。这需要对每一层的所有 GPU 进行聚合，进而需要高带宽

沟通。TP 对于同一服务器内通过高带宽互连 (e.g., NVLink [45]) 连接的 GPU 来说性能更好，而 PP 是跨服务器的首选。由于大多数开源模型 [34]、[38]、[67] 适合单个服务器中的 8 个 GPU，因此我们在本文的其余部分仅考虑 TP；这些想法可以很容易地扩展到 PP。我们将 2 个、4 个和 8 个 GPU 上的张量并行度分别表示为 TP2、TP4 和 TP8。

数据中心的电力和能源 大量工作探索了传统数据中心的电力/能源效率 [7]、[29]、[31]、[62]。然而，大语言模型快速增长带来了尚未得到广泛研究的新挑战。LLM 推理工作负载在数据中心负载中所占的比例迅速增加[49]。再加上为服务这些工作负载而部署的 DGX A100 和 H100 等功率密集型硬件，使其成为电力、能源和碳密集型产品 [12]、[49]、[58]。为了有效应对这一挑战，建立一个全面的框架来管理这些系统中的能源非常重要。

三. 提高能源效率的机会

为了了解 LLM 推理环境的能效特性，我们使用 vLLM [26] 推理引擎在 NVIDIA DGX H100 服务器 [44] 上表征开源模型 [33]、[38]、[39]、[66]。我们通过改变请求长度、请求负载、模型和服务 SLO 来分析 LLM 的能量属性。此外，我们使用 Azure 中的两个 LLM 服务 (Coding 和 Conversation) 的调用跟踪来分析实际生产环境中分析变量如何随时间变化。这些跟踪包括一周内分析服务收到的调用子集，并包含调用的时间戳以及输入和输出令牌的数量。这些跟踪是我们针对相同服务的开源跟踪的超集 [50]。

A. Heterogeneous Energy-Performance Profiles

请求长度 LLM 推理中的预填充和解码阶段表现出不同的执行行为 (第 II 节)，表明 *different input and output lengths* 的请求具有不同的计算和能量特征。我们根据输入/输出令牌的数量将请求分类为 9 个桶：SS (短输入，短输出)，SM (短输入，中输出)，SL (短输入，长输出)，MS，MM，ML，LS，LM 和 LL。表 IV 显示了阈值和相应的 TTFT/TBT SLO。我们使用来自 Azure 的 Conversation 服务的跟踪的输入/输出长度的 33rd、66th 和 100th 百分位数设置请求长度的阈值。我们将 SLO 设置为 5× 系统上独立运行的单个请求的延迟 [30]。

我们使用这些类别来表征在各种 GPU 频率和模型并行度下运行 Llama2-70B [33] 模型的不同请求类型的能耗，其中系统负载为每秒 2000 个令牌 (TPS)。表一以热图的形式显示了我们的结果。由于较短的请求不是计算密集型的，因此它们可以通过任何张量并行性来满足 SLO，并且通常以较低的速度

Tensor Parallelism		TP2				TP4				TP8			
GPU Frequency (GHz)		0.8	1.2	1.6	2.0	0.8	1.2	1.6	2.0	0.8	1.2	1.6	2.0
Input	Output												
Short	Short		0.77	0.97	1.03	0.94	0.79	0.91	1.01	1.35	1.19	1.29	1.49
Short	Medium		2.78	3.45	3.68	3.39	2.82	3.37	3.81	4.55	4.15	4.43	4.74
Short	Long					4.84	4.17	4.97	5.52	6.37	5.62	5.59	6.95
Medium	Short			1.02	1.09		1.08	1.07	1.20	1.51	1.29	1.34	1.73
Medium	Medium						4.23	3.91	4.08	5.34	4.39	4.56	5.44
Medium	Long						4.99	4.66	4.53	6.86	5.79	6.52	7.12
Long	Short						1.51	1.64	1.76	2.55	2.53	2.83	2.94
Long	Medium										7.71	8.81	9.17
Long	Long										12.99	11.89	13.21

表 I: Llama2-70B 在中等系统负载（每秒 2K 令牌）下改变请求长度、频率和模型并行性时的能耗（以瓦特小时 (Wh) 为单位）。违反 SLO 的配置显示为空灰色框，而可接受的配置根据每行的能耗以热图的颜色显示。

Tensor Parallelism		TP2				TP4				TP8			
GPU Frequency (GHz)		0.8	1.2	1.6	2.0	0.8	1.2	1.6	2.0	0.8	1.2	1.6	2.0
Low Load				3.41	3.75	3.44	2.93	3.71	3.73	4.49	3.76	4.52	4.64
Medium Load							4.23	3.91	4.08	5.34	4.39	4.56	5.44
High Load								4.22	4.13	5.86	5.24	5.42	6.62

表 II: LLama2-70B 中型输入和输出 (MM) 的能耗（以 Wh 为单位）在不同系统负载下要求不同的频率和模型并行性：低 (650 TPS)、中 (2K TPS) 和高 (4K TPS)。

Tensor Parallelism		TP2				TP4				TP8			
GPU Frequency (GHz)		0.8	1.2	1.6	2.0	0.8	1.2	1.6	2.0	0.8	1.2	1.6	2.0
Llama2-13B [32]		1.05	0.99	1.14	1.24	1.52	1.27	1.58	1.65	2.61	2.35	2.74	3.45
Mixtral-8x7B [39]		1.03	0.98	1.21	1.32	1.39	1.51	2.09	2.31	2.57	3.06	3.71	4.66
Llama2-70B [33]							4.23	3.91	4.08	5.34	4.39	4.56	5.44
Llama3-70B [34]							4.32	4.28	4.57	6.11	5.18	5.42	6.45
Mixtral-8x22B [38]										3.83	3.23	3.65	4.03
Falcon-180B [66]										9.56	7.94	8.57	10.34

表 III: 不同 LLM 架构的中等规模 (MM) 请求的能耗（以 Wh 为单位），改变频率和模型并行性以及中等系统负载 (2K TPS)。

	Input	Output	TTFT SLO	TBT SLO
Short	S	<256	<100	250 ms
Medium	M	<1024	<350	400 ms
Long	L	≤8192	≥350	2000 ms

表 IV: 基于输入/输出长度和相应的 TTFT/TBT SLO 对请求进行分类的阈值。

与其他频率相比。例如，SS 请求的最低能量配置是 1.2 GHz 的 TP2。相反，LL 请求只能在不违反 SLO 的情况下与 TP8 一起运行。对于 TP8，LL 请求的最低能量配置是 1.6 GHz。请注意，由于执行时间增加，满足 SLO (TP8, 1.2 GHz) 的最低功耗配置并不是能量最佳配置。一起运行所有请求将要求系统以最受约束的 SLO 配置运行，在本例中，按照 LL 配置运行。这将使系统能源效率低下。

为了利用这种异构性来提高能源效率，系统需要根据输入/输出长度来分离请求，并使用不同的服务器配置处理不同的请求类型。然而，在请求到达时，输入长度是已知的，但是，由于自回归 LLM 的性质，

输出长度未知。因此，系统需要预测输出长度。Dynamo LLM 将依赖于先前的工作，以相对较高的精度有效地执行此类操作 [19]、[55]、[79]，并将具有减轻偶尔错误预测影响的机制。

请求负载除了请求长度之外，LLM 推理服务器的传入负载也决定了计算需求。在低负载期间，系统有更大的 SLO 余量可供利用，并且可以以低频配置运行请求以节省能源。相反，在高负载期间，系统没有足够的 SLO 裕量，需要以高频配置运行。

表 II 显示了在改变每秒处理的提示令牌数 (TPS) 的情况下运行 Llama2-70B 中等大小的输入和输出 (MM) 请求时消耗的能量。该系统可以在几乎任何频率下以任何 TP 低负载运行。在所有可行的配置中，能量最低的配置是 1.2 GHz 的 TP4。TP8 需要更多 GPU 并行运行，因此会消耗更多能量。TP2 使用更少的 GPU，但增加了执行时间，并迫使各个 GPU 以高频率运行以满足 SLO，从而导致高能耗。相反，在高负载下，

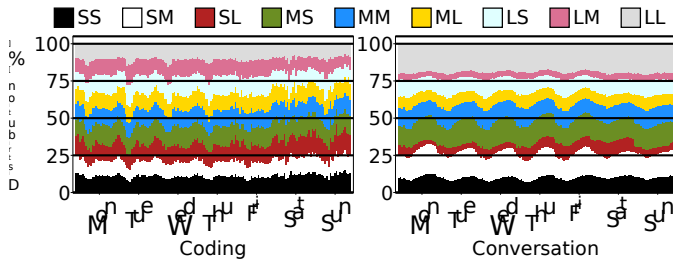


图 1: 根据输入和输出长度的请求分布, 分为三组: 短、中和长。

系统无法在 TP2 上运行, 需要 TP4 或 TP8。最低能量配置是 2 GHz 的 TP4。总的来说, 为了在性能限制下运行时最大限度地减少能耗, 我们需要考虑传入负载以设置正确的并行度和 GPU 频率。

要求的模型 **大语言模型** 属性的多样性直接转化为其能源概况。表 III 显示了在中等系统负载下运行中等大小的请求时不同 LLM 的能耗。较小的模型, 例如 Llama2-13B 和 Mixtral-8x7B, 可以与任何 TP 一起运行 (甚至使用单个 GPU); 它们的最低能量配置是 1.2 GHz 的 TP2。Mixtral-8x22B 和 Falcon-180B 要大得多, 只能与 TP8 一起运行。它们的最低能量配置是 1.2 GHz 的 TP8。

具有大量参数的计算限制模型对 GPU 频率和模型并行性更加敏感。因此, 它们通常需要在高频和高能量模式下运行。参数数量相对较少的稀疏模型可以承受较低的频率和较低的模型并行性。因此, 即使在性能较低的模式下, 它们也能满足性能要求。

服务 SLO 不同的服务通常使用具有不同 SLO 要求的相同模型 [57]。如前所述, 我们假设 SLO 使得 P99 尾部延迟在卸载系统上请求执行时间的 $5\times$ 之内 [30]。然而, 某些服务的 SLO 更为宽松, 单个请求执行次数为 $10\times$ 甚至 $20\times$ [10]、[37]。对于不同的 SLO 要求, 系统可能需要不同的能量优化配置。例如, 表 I 显示, 在严格的 SLO ($5\times$) 下, 中等负载下的短输入长输出大小的 Llama2-70B 请求在 TP4 和 1.2GHz 下具有最佳配置。然而, 如果采用宽松的 SLO ($10\times$), 请求甚至可以在 1.6GHz 的 TP2 上运行。

洞察#1 LLM 工作负载的能源性能概况高度异构。为了在性能 SLO 下实现最佳能源, 需要单独且不同地处理不同的请求 (尺寸、型号和 SLO)。

B. Dynamic LLM Inference Workloads

更改请求长度分布 我们测量了 Coding 和 Conversation 服务的请求类型的分布。图 1 显示了一周内每个工作负载的请求分布。不同服务的分布有所不同。Conversation 通常具有较长的输出和较短的输入, 而

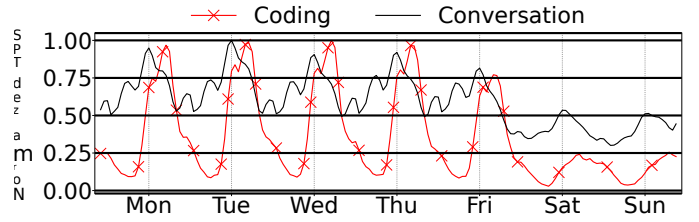


图 2: Coding 和 Conversation LLM 推理工作负载一周内的负载。

Coding 显示出相反的趋势。然而, 这两种服务在每种请求类型中都占很大一部分, 而且重要的是, 请求类型的受欢迎程度随着时间的推移而变化。

正如前面所观察到的, 不同的请求类型需要不同的能源优化配置。因此, 系统需要将其资源划分为每个请求类型的池, 单独配置池, 并根据当前请求分布动态调整池的配置。然而, 如果系统将请求分为太少的类别, 它将无法微调系统以获得最佳能量。另一方面, 太多的类可能会导致碎片并对能源效率产生负面影响。因此, 系统必须找到合适数量的资源池。在 DynamoLLM 中, 我们将使用历史数据来设置池的数量, 以便具有不同 SLO 要求 (TTFT 或 TBT 限制) 和计算属性 (计算或内存限制) 的请求具有单独的池。此外, 随着给定请求类型的负载减少, DynamoLLM 将通过将该池与服务更长请求的下一个可用池合并来避免碎片。

请求负载波动 LLM 推理工作负载作为面向用户的应用程序, 表现出典型的昼夜模式, 在工作时间达到峰值, 在夜间和周末达到谷值。图 2 显示了一周内两个工作负载的每秒令牌负载。负载被标准化为各个工作负载的峰值。Coding 轨迹显示出清晰的昼夜模式, 每天都有峰值, 夜间负载较低, 周末负载低得多。Conversation 显示了一个不太极端但仍然重要的昼夜模式。

Conversation 的峰值负载分别比其平均负载和谷值负载高 $1.7\times$ 和 $3.3\times$ 。Coding 的峰值负载分别比其平均负载和谷值负载高 $2.8\times$ 和 $34.6\times$ 。这种大的松弛表明 LLM 推理服务器可以经常在性能较低但能源优化的配置下运行, 而不会违反 SLO。一旦负载开始增加, 服务器就需要切换到性能更高的操作模式。

LLM 服务 SLO 和模型多样性最后, 不同的服务可以分时共享相同的 LLM 模型实例 [14]。它们可能具有不同的 SLO, 需要根据当前服务用户调整配置。另一方面, 同一服务可能同时使用多个不同的模型 [11]。这需要不同的执行计划来实现各个查询的最佳能耗。因此, 对于服务提供商来说, 在满足性能 SLO 的同时在能源优化环境中运行并非易事。

Overhead source	Time
Create a new H100 VM [36]	~1-2 min
Initialize distributed multi-GPU environment	~2 min
Download model weights (Llama2-70B [67])	~3 min
Set up the engine configuration	~18 sec
Install weights and KV cache on GPUs	~15 sec
Total	~6-8 min

表 V: 创建 LLM 推理服务器 VM 的新 8×H100 实例的测量开销。

见解 #2 LLM 工作负载是高度动态的, 因此, 能源优化配置很快就会变得次优。然而, 大搜索空间的复杂性需要自动且用户透明的配置选择。

C. Reconfiguration Overheads

为了捕捉 LLM 推理工作负载的快速变化, 我们需要在配置之间快速转换。但是, 更改 (1) 推理服务器实例数量、(2) 模型并行性和 (3) GPU 频率会产生开销。

更改实例数量 为了适应波动的负载, 动态调整 LLM 实例的数量来服务请求 (*i.e.*, 缩小和缩小) 是具有成本效益的。然而, 在推理负载的关键路径上添加新的推理服务器的开销太大, 难以承受。表 V 显示了以下开销的细分: (1) 在云中实例化新的 GPU VM (例如 H100 VM [36]), (2) 初始化分布式多 GPU 环境 (*e.g.*, Ray、MPI), (3) 下载模型权重, (4) 设置推理引擎, 以及 (5) 在 GPU 上安装权重和键值缓存。总的来说, 这些开销甚至可能需要几分钟。因此, 传统的 LLM 推理环境通常会提供静态数量的实例来处理其峰值负载, 从而导致利用率严重不足。在 DynamoLLM 中, 我们将提出有效扩展实例数量 (在当前负载下) 的技术, 同时最大限度地减少大部分横向扩展开销。

更改模型并行度 要修改 LLM 推理服务器的模型并行度, 我们需要执行两个操作。首先, 我们需要重新分片模型权重并将其传输到正确 GPU 的内存中。其次, 推理引擎需要同步所涉及的 GPU。当前系统停止引擎, 从 GPU 卸载权重, 将权重从主机加载到新的一组 GPU, 然后从头开始重新启动引擎。如果在关键路径上执行, 这会增加难以忍受的开销 (大约 1-2 分钟)。在 DynamoLLM 中, 我们将展示如何通过巧妙地将逻辑映射到物理 GPU、利用 GPU 间直接 NVLink 连接以及在后台在 GPU 之间移动权重来最小化重新分片开销。

更改 GPU 频率 设置 GPU 频率 (*e.g.*, 通过 `nvidia-smi` [46]) 会产生不可忽略的开销。它涉及调用操作系统、通过系统调用与 GPU 驱动程序通信以及通过固件执行硬件交互。平均而言, 设置 GPU 频率需要

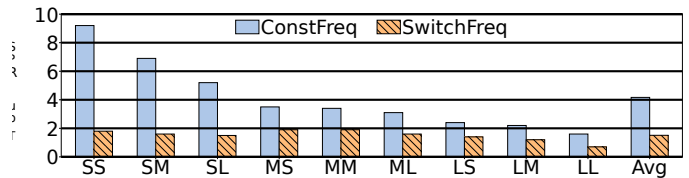


图 3: 恒定频率 (1980MHz) 以及在后台每次迭代时重新设置频率 (至 1980MHz) 的不同请求类型的吞吐量。

大约 50-80 毫秒。相比之下, LLM 推理过程的一次解码迭代需要 20-30 毫秒。因此, 调整 GPU 频率所花费的时间会显着影响整体性能, 可能会使各个推理步骤的延迟加倍。图 3 显示了在每次 LLM 推理迭代中以最高频率 (1980 MHz) 持续运行以及在后台重新设置频率 (至 1980 MHz) 时不同请求类型的吞吐量。由于软件开销, LLM 推理系统的吞吐量显着下降。因此, 优化或最小化 LLM 推理期间的频率变化对于保持高效和响应性能至关重要。见解 #3 在 LLM 服务器配置之间进行转换会产生大量开销。为了提高能源效率, 在计算能源/性能权衡时需要最小化并考虑此类开销。

四. DYNAMOLLM: LLM 推理集群的能源管理框架

我们利用这些见解来设计 DynamoLLM, 这是第一个适用于 LLM 推理环境的能源管理框架。DynamoLLM 与现有推理平台无缝集成, 使 LLM 工作负载能够高效节能且经济高效地运行, 同时满足其性能 SLO。DynamoLLM 有四个关键原则。首先, 它是 *energy-optimized and SLO-aware*, 利用模型配置文件在其 SLO 要求范围内自动为特定 LLM 和推理工作负载选择最节能的配置。其次, DynamoLLM 通过将集群资源划分为针对特定请求类型定制的 *instance pools* 来微调异构 LLM 工作负载的配置。第三, DynamoLLM 可以适应所选组织 *dynamically reconfiguring* 波动的 LLM 推理负载。最后, 为了确保频繁且顺利地重新配置, DynamoLLM *minimizes reconfiguration overheads*。

架构图 4 显示了 DynamoLLM 架构。该系统按集群、池和实例级别分层组织。在每个级别, 控制器都会调整其指定的配置旋钮, 并与上层和下层的控制器传达其决定。控制器使用分析阶段生成的能源性能模型来确定给定当前系统状态的能源优化操作的实例数量、模型并行化和 GPU 频率。 (1) *Cluster Manager* 接收推理请求, 预测其类型, 并将其转发到适当的实例池。此外, 它还围绕

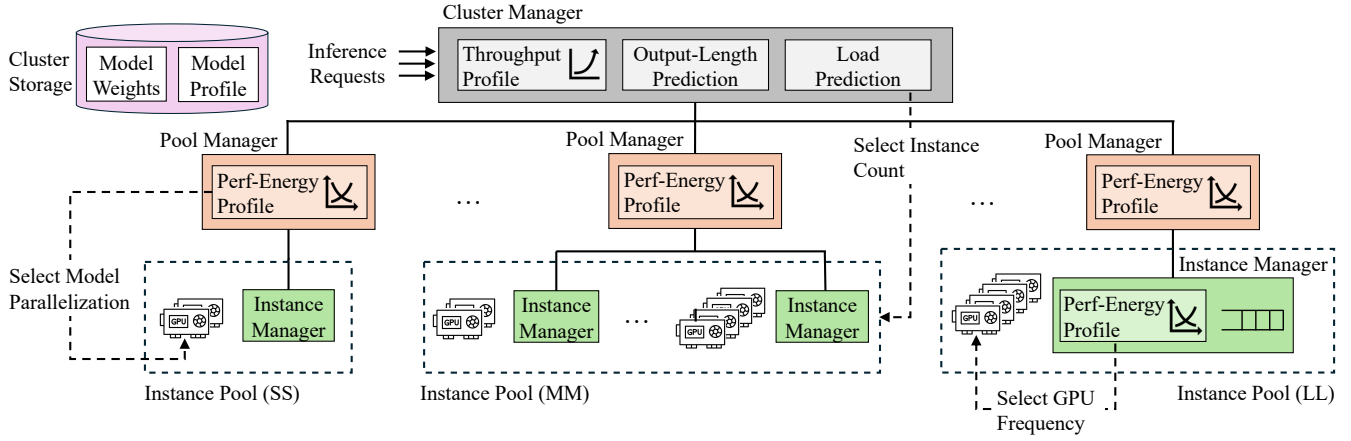


图 4: DynamoLLM 架构: 控制器层次结构, 集群资源按请求类型划分为池。

根据系统负载, 有规律地重新评估需要多少个池以及每个池需要多少个模型实例。(2) *Pool Manager* 以最小化每个池能耗的方式调度模型实例的请求。它还定期检查其实例是否需要重新分片为更节能的并行化。(3) *Instance Manager* 调度推理引擎的请求, 并定期检查实例的 GPU 频率是否需要调整。

A. Configuring Instances for Energy-Efficiency

生成 LLM 配置文件 将服务部署到 DynamoLLM 时, 用户指定服务使用的 LLM 和预期性能 SLO。然后, 系统表征模型并生成其能源性能概况。DynamoLLM 通过在不同模型并行度 (TP2、TP4 和 TP8) 和 GPU 频率 (800–1980M Hz, 步长为 200MHz) 下运行不同请求长度的负载来分析模型。系统会分析几个负载级别, 直至最大吞吐量, 然后推断测量到的负载之间的行为。分析结果是一个函数, 它将负载、请求长度、模型并行性和 GPU 频率作为输入, 并输出预期的能耗和 TTFT/TBT 延迟。

由于许多服务可能使用相同的模型, DynamoLLM 可以跨服务重用配置文件, 从而最大限度地减少分析开销。此类配置文件存储在全局 DynamoLLM 存储库中, 然后在集群中部署给定服务时缓存在集群本地存储中。

选择能源优化配置 给定当前负载和可用资源, DynamoLLM 使用生成的配置文件最大限度地减少能源消耗, 同时保持在性能限制范围内。系统将此任务表述为混合整数线性规划 (MILP) 求解器的优化问题。求解器需要输出每个张量并行性需要多少个实例 (N^{TP_2} 、 N^{TP_4} 和 N^{TP_8})、它们应以什么频率运行 (f^{TP_2} 、 f^{TP_4} 和 f^{TP_8})、以及应将哪个负载分配给每个实例 (L^{TP_2} 、 L^{TP_4} 和 L^{TP_8})。我们假设给定并行性的所有实例都以相同的频率运行并接收公平共享的工作量。

求解器的优化目标是最小化总能耗 (E), 约束条件为: 1) 所有实例类型使用的 GPU 总数不超过分配的 GPU 数量 (N); 2) 分配给各个实例的负载总计为总预期负载 (L); 3) 具有指定负载的所有实例的预期性能均符合要求 (SLO)。当以 f_i GPU 频率以 TP_i 并行度运行负载 L^{TP_i} 时, 函数 $Energy^{TP_i, f_i}(L^{TP_i})$ 和 $Performance^{TP_i, f_i}(L^{TP_i})$ 分别输出预期的能量和性能。那么, 优化任务可以表述为:

$$\begin{aligned}
 \min \quad & \left(\sum_i (N^{TP_i} \times Energy^{TP_i, f_i}(L^{TP_i})) \right) \quad \forall i \in \{2, 4, 8\} \\
 \text{s.t.} \quad & \sum_i i \times N^{TP_i} \leq N \\
 & \sum_i (N^{TP_i} \times L^{TP_i}) \geq L \\
 & Performance^{TP_i, f_i}(L^{TP_i}) \leq SLO
 \end{aligned} \tag{1}$$

这种方法保证了能量的优化配置。然而, 由于求解器的搜索空间很大, 它引入了不可忽略的开销 (*i.e.*, ~数百秒)。因此, 它不能用于以细粒度间隔 (*e.g.*, 每隔几秒) 选择正确的系统配置。接下来, 我们展示如何将任务分解为子任务的层次结构, 并使用近似启发式来降低计算复杂性。

B. Hierarchical Control for Dynamic Load

DynamoLLM 通过将特定的优化任务分配给各个控制器来简化计算。控制器不是搜索全局最优配置, 而是在上层控制器施加的约束下并假设下层控制器以最高性能配置运行的假设下为各个旋钮设置局部最优值。这使得控制器能够在不同的时间尺度上运行——从节点调整的分钟到频率调谐的秒。由于每次运营变化都涉及不同的管理费用和能源效率影响, 因此需要不同的规模。

横向扩展/缩减 在每个时期 (e.g., 30 分钟), 集群管理器都会计算每个池中可以支持给定请求类型负载的最小节点数。管理器使用 *load predictor* 根据其历史数据 (e.g., 通过轻量级负载模板 [62]) 预测每种请求类型的传入负载 PL 。此外, 管理器假设所有实例都将以最高性能配置运行 (i.e., TP8, 1980 MHz)。然后, 如果给定请求类型的预测峰值为 PL , 并且节点可以支持该请求类型的最大负载为 ML , 则管理器将 $\lceil \frac{PL}{ML} \rceil$ 节点分配给该池。通过将工作整合到少量节点中, 系统试图最大限度地降低用户的成本和轻负载 GPU 的闲置能量。

Handling fragmentation: 分配资源来处理峰值负载可能会导致资源利用率不足。如果池中的过度配置不断累积, 能源效率就会下降。为了防止出现此类情况, DynamoLLM 会向给定实例池少分配一个实例, 并将剩余负载移至下一个较大请求类型的实例池。集群管理器使用此信息在下一个调度周期 (e.g., 30 分钟) 期间将给定请求类型的一部分负载转发到适当的较大实例池。通过这种方式, 只有最大请求的实例池才能被过度配置, 从而最大限度地减少集群级别的碎片。

在每个时期 (e.g., 每 5 分钟), 池管理器决定如何将集群管理器分配的 N GPU 拆分为正确的模型并行性 (在池中创建多少个实例) 和张量并行性 (每个实例使用多少个 GPU)。池管理器使用公式 (1) 的简化版本, 假设所有实例都以最高 GPU 频率 (i.e., 1980 MHz) 运行。因此, 我们的目标是最大限度地减少能耗, 同时以最高频率运行固定数量的 GPU, 并仅控制并行度旋钮。

Accounting for the overheads: DynamoLLM 将转换开销 (向外扩展/向内扩展、向上分片/向下分片) 存储在 *Overhead Table* 中。该表与控制器集成, 以便在计算新配置的能源效益时, 可以考虑重新配置的成本。控制器评估重新配置所节省的能源是否值得相关的管理费用和停机时间。

Reducing downtime: 由于存在长时间停机的风险, 重新配置无法在所有实例上同时进行。相反, DynamoLLM 采用交错重新配置方法, 一次重新配置实例的子集 (e.g., 重新分片)。这确保了在某些实例正在进行重新配置时, 其他实例仍保持运行以处理持续的工作负载。系统首先重新配置具有较高节能潜力的实例。

向上/向下扩展 最后, 在每个时期 (e.g., 5s), 实例管理器都会根据指定的模型并行性微调 GPU 频率, 以进一步节省能源。实例管理器使用性能配置文件首先过滤掉当前负载下违反 SLO 的频率。然后, 它使用能量配置文件来选择可接受的频率, 以最大限度地减少能量消耗。

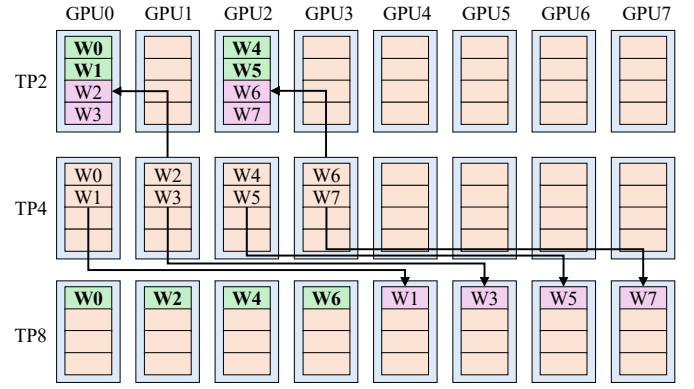


图 5: 将 TP4 模型重新分片为 TP2/TP8 的示例。

C. Reduced Overheads for Smooth Reconfiguration

为了实现频繁的重新配置, DynamoLLM 提出了一组技术来最小化以下开销: (1) *scaling- in/out* LLM 推理服务器的数量, (2) *sharding- up/down* 给定实例的并行性, 以及 (3) *scaling- up/down* 给定实例的 GPU 频率。

扩展/扩展推理服务器 DynamoLLM 通过实施多种策略减少了创建新服务器实例的开销。首先, 它将模型权重保留在集群内本地缓存 (如图 4 所示), 从而无需从全局存储库中获取它们。其次, 它从快照启动虚拟机, 推理引擎的整个状态已经初始化, 从而减少了启动时间。该快照包括预加载的库、GPU 驱动程序和推理引擎配置。第三, 它在主动工作负载处理的关键路径之外主动在后台创建新的虚拟机。具体来说, DynamoLLM 会预测下一个调度周期的峰值负载, 并在该周期开始之前启动额外的 VM。通过让这些虚拟机随时可用, DynamoLLM 可以将一部分负载无缝卸载到新实例, 而不会产生任何明显的延迟影响。

对实例进行上/下分片 为了减少重新分片的开销, DynamoLLM 优化了 GPU 上的权重分配。我们提出了两种技术来最大限度地减少数据传输和单个传输的延迟。首先, 系统开发了一种图形匹配算法, 可最大限度地增加当前 GPU 中保持静止的权重数量。该算法将当前权重分布和所需的张量并行度作为输入, 并输出源和目标 GPU 以及要在每个源-目标对之间传输的权重分数。具体来说, 该算法构建了一个二分图, 其中节点代表当前和下一个配置中的 GPU。节点之间的边代表潜在的传输, 按要传输的数据量加权。然后, 它应用最大权重匹配算法来找到最小化边的总权重的最佳传输计划 (i.e., 最小化传输的数据量)。其次, 为了减少传输延迟, 系统通过 NVLink 使用 GPU 间直接传输, 允许它们将其权重的一部分并行发送到其他 GPU, 而无需任何主机干预。

图5显示了从TP4重新分片到TP2和TP8的示例。首先考虑从较低级别并行性到较高级别并行性（TP4→TP8）时的情况。在初始状态（TP4）下，GPU 0-3 各持有四分之一的模型权重。在最终状态（TP8）中，所有 GPU 都需要承受模型权重的 8 倍。因此，前四个 GPU 已经具有其所需的状态，它们只需将一半的权重发送给其余四个 GPU。四个传输（e.g., GPU0→GPU4, GPU1→GPU5, ...）并行发生。因此，重新分片需要时间通过 NVLink 发送 1/8 的模型权重（在我们的设置中大约为 50 毫秒）。

现在考虑从较高并行度到较低并行度（TP4→TP2）的情况。在最终状态（TP2）中，两个 GPU 需要各自持有一半的权重。由于每个 GPU 最初持有四分之一的权重，因此我们将两个 GPU 的权重合并为一个 GPU：GPU1 将权重 W2/W3 发送到 GPU0，GPU3 将权重 W6/W7 发送到 GPU 2。由于这两个传输并行发生，因此总的重新分片时间是发送 1/4 模型权重的时间（在我们的设置中约为 100 毫秒）。表 VI 显示了采用我们的优化方法时不同源/目标对的重新分片开销。一些配置快速切换到其他配置（转换时间为0），其他变化会产生较大的开销（转换时间为4T，其中T是发送1/8权重的时间）。

此外，在某些转换中，实例继续以相同的吞吐量为请求提供服务。从较小的张量并行度缩放到较大的张量并行度（e.g., TP4→TP8）时就是这种情况。旧实例将一部分权重发送到新实例，而不增加其内存占用。在其他转换期间，当前实例需要在较低吞吐量下运行。当实例从较大张量并行度扩展到较小张量并行度（e.g., TP8→TP4）时就会出现这种情况。旧实例使用的一些 GPU 接受额外的权重，从而减少其内存容量来满足新的请求。一般来说，每当保存模型权重所需的 GPU 内存增加时，由于传入请求的内存容量较低，吞吐量就会下降。

最后，将权重发送到正确的内存后，推理引擎需要同步运行新实例的 GPU。最先进的引擎，例如 vLLM [26]，可以在几百毫秒到几秒内执行此操作。在此期间，实例无法接收任何负载，从而导致明显的停机时间。为了减少停机时间，DynamoLLM 允许旧实例在新实例执行同步过程时处理请求。仅当新实例上线时，旧实例才会被删除。仅当新旧实例的内存总和低于 GPU 的内存容量时，才有可能实现这一点。当总和超过内存容量（e.g., TP2和TP4，参数为70B）时，需要先关闭旧实例，然后再创建新实例。总体而言，不同的转换会产生不同的开销和实例停机时间，需要将一部分负载转移到另一个实例。DynamoLLM 最大限度地减少了开销，并在决定是否重新配置实例时考虑其对整体效率的影响。

Src/Dst	TP2	4TP2	TP4	TP2+TP4	2TP4	TP8
TP2	0	4T	2T	2T	2T	T
4TP2	0	0	0	0	0	0
TP4	2T	2T	0	2T	2T	T
TP2+TP4	0	2T	0	0	T	T
2TP4	T	T	0	T	0	0
TP8	T	T	T	T	T	0

表 VI: 重新分片时传输模型权重的开销。T 是移动模型 1/8 的时间单位（e.g., NVIDIA DGX H100 [44] 和 Llama2-70B 模型 [67] 上的 NVLink 带宽为 300GB/s, $T = 50ms$ ）。

调高/调低频率 通过将 NVIDIA 系统管理接口 (nvidia-smi) 监视器程序直接加载到内存中，可以最大限度地减少更改 GPU 频率的开销。这样就无需在每次需要调整频率时重新加载程序。此外，通过在特权模式下运行控制器，DynamoLLM 避免了与操作系统-用户交互相关的开销，从而允许快速调整频率。

D. Predictive Scheduling for Request Heterogeneity

为了将请求的异构性映射到异构实例池，DynamoLLM 中的集群控制器使用 *output-length predictor* 来预测请求类型并将请求引导到正确的实例池。预测器充当代理模型，接受输入提示并将输出分类为短、中或长。基于预测的输出长度和已知的输入长度，集群管理器将请求转发给负责给定请求类型的池管理器。如果实例池当前过载，集群管理器会将请求转发到下一个可用的池以获取更大的请求类型。一旦请求到达正确的池，池管理器就需要从池选择一个实例。具体来说，在可能向该实例添加新请求之后，管理器使用分析步骤生成的模型来预测每个实例的能量和响应时间。然后，它选择总能耗最小化的实例，同时保持在 SLO 确定的每个实例吞吐量范围内。处理错误预测如果系统高估了请求长度，请求将被路由到性能更高的池。因此，它以次优能量运行，但其延迟不受影响。相反，如果请求长度被低估，则该请求将被放置到性能较低的池中，可能会丢失其 SLO。同样，负载预测错误可能会导致请求突发期间给定池的资源不足。在这两种情况下，由于一些错误预测，系统需要对创建的 *emergency event* 做出反应。

当实例管理器检测到其队列正在建立时，表明请求处理速率低于请求到达速率，它会触发紧急事件。首先，实例管理器尝试对其队列中的请求重新排序，并对那些即将错过截止时间的请求进行优先级排序。其次，如果某些请求即使在重新排序后仍会错过最后期限，实例管理器会将其 GPU 的频率提高到最大值。第三，如果

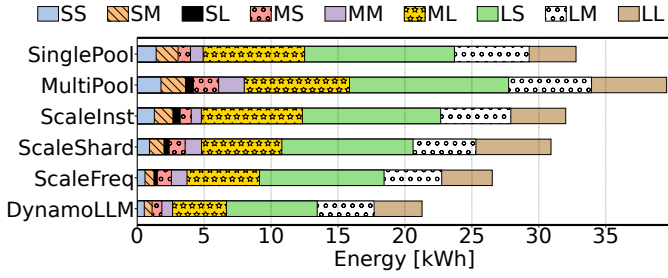


图 6: 运行开源 Llama2-70B 模型 [67] 的六个评估系统的能耗以及 1 小时开源生产跟踪 [50]。

积压持续存在或恶化，实例管理器会重新引导一些尚未开始执行的请求。请求的子集通过池管理器移动到池中的另一个实例。最后，如果所有尝试都不够，实例管理器就会采取更严厉的措施。其中一项措施涉及压缩等待处理超过指定阈值的请求。此操作会指示用户重试其请求，从而允许前端系统将这些请求重定向到备用实例池，或者稍后在系统负载稳定后重试。

E. DynamoLLM Implementation

我们在 vLLM [26]（最先进的 LLM 推理平台）之上构建 DynamoLLM。然而，DynamoLLM 的模块化使其无需修改即可与其他平台、e.g., TensorRT-LLM [47] 集成。我们将控制器实现为具有低内存和计算要求的轻量级 gRPC 服务器。集群和池管理器托管在专用虚拟机中，以实现稳健的管理。实例管理器与 LLM 推理引擎实例位于同一位置，以降低通信开销。对于输出长度预测，我们利用基于 BERT 的代理模型 [55]，它可以对传入请求进行准确有效的分类。对于负载预测，我们使用基于模板的方法，该方法使用历史数据对一周内的负载模式进行建模 [62]。池管理器使用 Python 的 PuLP 库 [53] 来解决 MILP。DynamoLLM 使用 SciPy [54] Python 库中的 *interp1d* 函数对能量和性能进行建模。

五、评价

A. Evaluation Setup

我们在具有 8 个 H100 GPU 的服务器上运行我们的实验 [44]。我们展示了 Llama2-70B [67] 的结果，但其他模型（i.e., Mixtral [38]、Falcon [66]、BLOOM [59]）也遵循相同的趋势。我们使用生产级跟踪设置负载：1 小时开源跟踪 [50] 以及来自我们队列的 *Coding* 和 *Conversation* 的 1 天和 1 周跟踪。我们将 DynamoLLM 与五个系统进行比较。SinglePool（实践状态基准）将所有请求调度到以最高 GPU 频率运行 TP8 的公共实例池。MultiPool 将 LLM 实例分隔在多个每个请求类型池中。另外还有 ScaleInst、ScaleShard 和 ScaleFreq

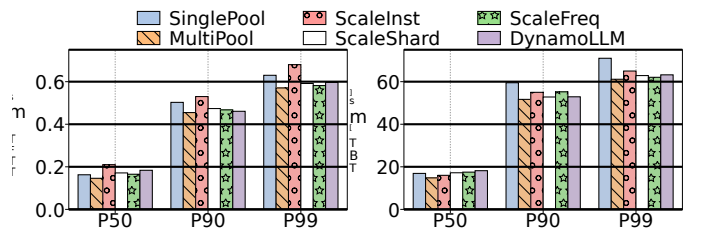


图 7: 运行开源 Llama2-70B 模型 [67] 的每个系统的延迟摘要，具有 1 小时开源生产跟踪 [50]。

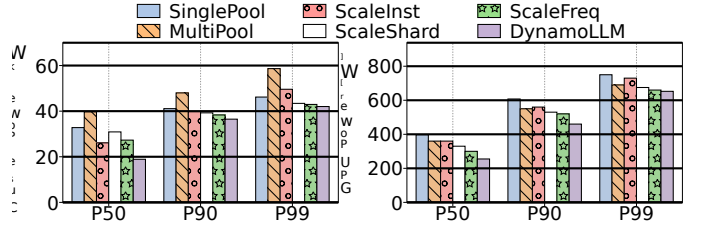


图 8: 运行开源 Llama2-70B 模型 [67] 的每个系统的功率摘要，具有 1 小时开源生产跟踪 [50]。

根据当前负载分别缩放池中的实例数量、模型并行度或 GPU 频率。

B. Cluster-Level Experiments

我们首先使用 *Conversation* 服务的 1 小时开源生产跟踪在 GPU 服务器集群上评估系统 [50]。我们为基线配置了 12 台 H100 服务器来处理峰值负载，而 DynamoLLM 根据当前负载扩展服务器数量。

能源 图 6 显示了实验中集群的能源消耗。MultiPool 比 SinglePool 增加了 20% 的能耗，因为它分配更多资源，同时始终以最高性能配置运行。同时，ScaleInst、ScaleShard、ScaleFreq 和 DynamoLLM 分别降低了能耗 4.1%、7%、19% 和 35%。Scale-Inst/Shard/Freq 通过配置一个旋钮来减少能源，但为进一步节省留出大量空间。最后，DynamoLLM 同步缩放多个旋钮以实现最低的能耗。我们进一步细分每个请求类型的总能量。图 6 显示较长的请求（e.g., LL）和高度流行的请求（e.g., ML）比其他类型消耗更多的能量。

延迟 图 7 显示了每个系统的 TTFT/TBT 延迟。通过将请求类型分离到不同的资源池中，MultiPool 消除了队头阻塞效应，并减少了 SinglePool 的延迟。同样，ScaleShard 和 ScaleFreq 以及 DynamoLLM 减少了尾部延迟。然而，当存在可用的 SLO 裕量时，这些系统会在较低性能模式下运行，从而略微增加 P50 延迟。另一方面，由于在用户负载的关键路径上创建新的推理服务器的巨大开销，ScaleInst 增加了尾部延迟。总体而言，DynamoLLM 减少了

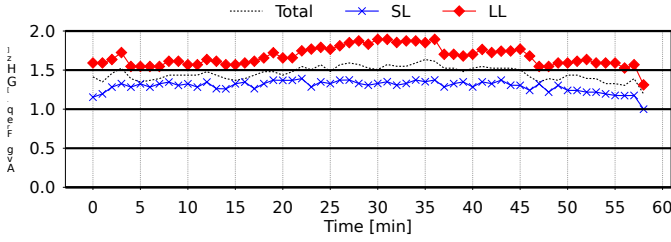


图 9: 运行开源 Llama2-70B 模型 [67] 的 DynamoLLM 一小时内的 GPU 频率, 以及 1 小时开源生产跟踪 [50]。

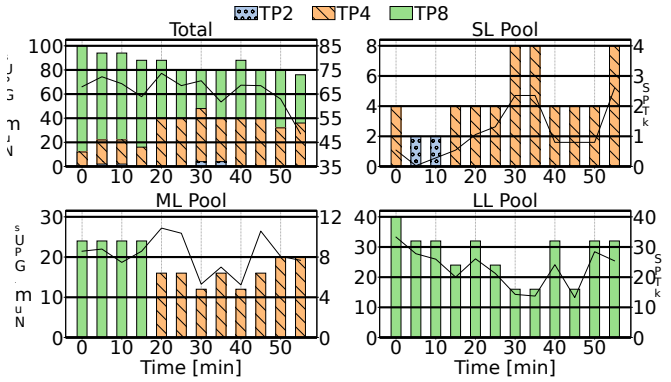


图 10: 随着时间的推移, 运行开源 Llama2-70B 模型 [67] 以及 1 小时开源生产跟踪 [50], 每个分片配置 (TP2、TP4 或 TP8) 使用的 GPU 数量。

与 SinglePool 相比, P99 TTFT 和 TBT 延迟分别增加了 5.3% 和 11.1%, 而 P50 TTFT 和 TBT 延迟分别增加了 11.4% 和 7.6%。

功耗图 8 显示了整个集群系统的功耗 (右图) 和每个 GPU 的平均功耗 (左图)。由于在节能模式下运行, DynamoLLM 有效降低了集群和每个 GPU 的功耗。DynamoLLM 使 P50 和 P99 功耗较基准分别降低了 43% 和 9%。

频率变化图 9 显示了 (1) 整个集群、(2) 服务短请求的池和 (3) 服务长请求的池随时间变化的平均 GPU 频率。平均频率始终显著低于基线使用的最大允许频率 (1980 MHz)。DynamoLLM 通过以不同的频率操作池来有效地适应不同的请求类型。

分片更改图 10 显示了整个集群和各个池 (SL、ML 和 LL) 的不同模型并行性 (TP2、TP4 和 TP8) 所使用的 GPU 数量。该图还显示了给定池随时间变化的负载情况。不同的池以不同的模型并行度运行, DynamoLLM 可以随着负载的变化有效地改变并行度。

C. Sensitivity Studies

对预测器准确性的敏感性我们分析预测模型的准确性如何影响整个系统

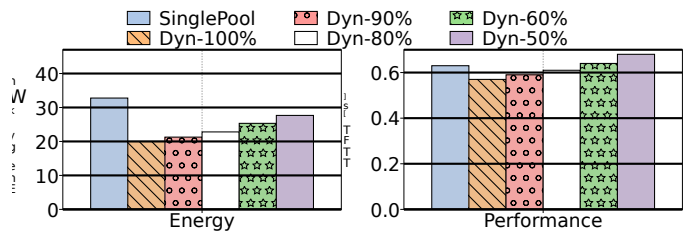


图 11: 运行开源 Llama2-70B 模型 [67] 时不同精度的能量和性能, 具有 1 小时开源生产轨迹 [50]。

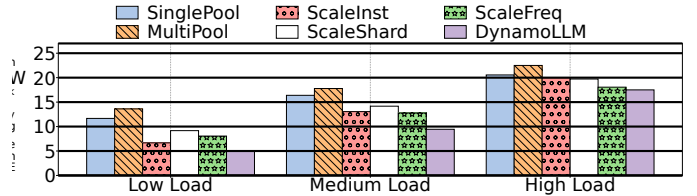


图 12: 运行开源 Llama2-70B 模型 [67] 时不同负载级别的能量。

效率。我们引入了输出长度错误分类的有界误差, 并测量了中等负载的能耗。图 11 显示预测器精度对能源和性能的影响都很小。与没有错误的环境相比, 有 40% 错误的环境会导致能耗增加 13%, TTFT 增加 7.3%。对预测错误具有鲁棒性的原因是 DynamoLLM 可以及时检测错误预测并相应地重新配置旋钮。

对负载的敏感性 我们使用不同的系统负载来评估 DynamoLLM。我们使用泊松分布来生成低、中和高负载, 以获取请求到达间隔时间。图 12 显示了五个评估系统在不同负载水平下的能耗。在低、中和高负载下, DynamoLLM 分别将 SinglePool 基线的能量降低了 51%、40% 和 23.4%。随着负载的增加, DynamoLLM 的节能效果会降低, 因为系统更频繁地需要以更高的频率运行, 并具有更高水平的模型并行性。

对池数量的敏感性 图 13 显示了具有不同请求池数量的 DynamoLLM 的能耗和性能 (TTFT)。回想一下, 我们选择的设计有 9 个水池。如果添加太多池 (12 或 16 个), 系统就会变得碎片化, GPU 的闲置能量会导致整体能量增加。减少池的数量 (2 或 4) 可以防止系统针对特定请求类型微调频率和模型并行性。通过添加适当更多的池可以提高性能, 因为它有助于消除队头阻塞并引入更多的执行资源。

D. Long Cluster-Level Experiments

我们通过运行 Conversation 服务的 1 天跟踪来运行更长的实验。跟踪涵盖了所有调用

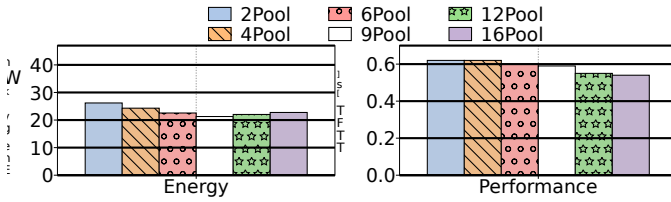


图 13: 运行开源 Llama2-70B 模型 [67] 的不同数量的池（或请求类型）的能量和性能，以及 1 小时开源生产跟踪 [50]。

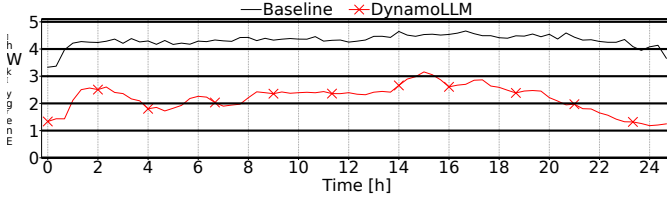


图 15: 运行开源 Llama2-70B 模型 [67] 的 SinglePool 和 DynamoLLM 的能耗以及 1 天生产跟踪。

典型工作日期间服务实例的子集。我们使用 SinglePool 在 11 台 H100 服务器上运行了 24 小时实验，并根据 DynamoLLM 中的负载扩展实例数量。图 15 显示了两个系统在 5 分钟时间间隔内的能耗。DynamoLLM 在高峰时段（当动态功率占主导地位时）和低利用率时段（当空闲功率占主导地位时）将能耗降低到基线之上。一整天，DynamoLLM 的能耗降低了 42%。

E. Large-Scale Simulations

为了将我们的见解推广到大规模，我们开发了一个离散时间模拟器，可以使用生产轨迹来模拟不同系统的能源消耗。图 14 显示了使用 *Conversation* 和 *Coding* 服务的 1 周跟踪记录的五个评估系统的标准化能耗。DynamoLLM 显著降低了两种类型服务的能源消耗。由于 *Conversation* 服务的输入长度通常较短（ML 主导请求类型），DynamoLLM 以更高能效模式运行。另一方面，*Coding* 服务在夜间和周末有深谷。因此，DynamoLLM 利用低负载时段来节省能源。DynamoLLM 将 *Conversation* 和 *Coding* 服务的能耗分别比基准降低了 47% 和 56%。

F. Cost and Carbon Emission

用户成本 DynamoLLM 通过最大限度地减少 GPU 数量并优化其能源效率来降低用户的运营成本。为期一周的实验的 GPU 服务器数量平均从 40 台减少到 24.6 台（成本降低 38.5%）。使用当前的 GPU VM 定价 [8]，这可以节省 136 2.7 美元/小时。通过减少能源消耗，DynamoLLM 将相关能源成本降低了多达

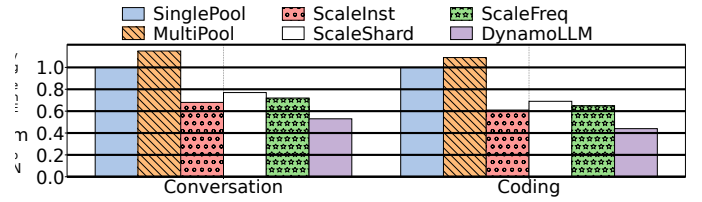


图 14: 六个评估系统的标准化能耗，以及运行开源 Llama 2-70B 模型的为期一周的生产轨迹 [67]。

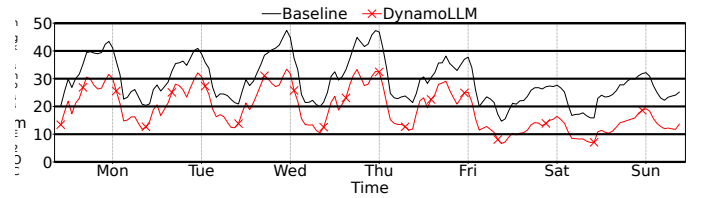


图 16: 使用运行开源 Llama2-70B 模型的 DynamoLLM 和 SinglePool 基线进行为期一周的对话跟踪，随时间变化的碳排放量 [67]。

56%。由于能源成本 [28] 目前远低于 GPU 成本，这意味着每小时仅节省 4.4 美元。碳排放 能源消耗转化为运营 CO_2 排放量。我们使用来自多个网格的长达一周的碳强度轨迹 [2]，并将碳强度映射到 SinglePool 基线和 DynamoLLM 随时间推移的能源消耗。图 16 显示了 CAISO 的两个系统随时间变化的运营碳排放量 [1]。SinglePool 和 DynamoLLM 每周消耗 5t 和 3.1t 的 CO_2 。这些可观的节省 (38%) 朝着可持续的大语言模型迈出了一步。

六. 相关工作

集群资源和电源管理大量的工作试图通过有效的资源共享 [6]、[43]、[51]、动态分配 [71] 和硬件重新配置 [24]，通过对各种延迟敏感的工作负载（例如微服务 [76] 和 IDL 工作负载）进行资源管理，在 SLO 约束下提高资源效率。其他人则关注利用工作负载特征 [25]、[75] 和系统状态 [62] 实现安全电源管理和超额订阅 [16]、[29]、[49] 的方法。

节能工作负载 先前的工作重点关注 CPU 工作负载的能源效率 [15]、[31]、[43]、[61]，研究人员开始探索 GPU 工作负载的独特能源属性 [58]、[65]、[74]。最近的方案建立在顶部，通过频率缩放 [20]、[23]、[40]、[41]、[64]、[77]、[82]、自动缩放 [22] 以及资源分区和映射 [18]、[64] 来管理 DNN 推理和训练的能源和功耗 [69]、[70]、[72]。我们表明，提高 LLM 推理的能源效率需要全面了解所有可用的旋钮。DynamoLLM 是一个整体框架，考虑到请求和负载的多样性和动态性，动态地重新配置所有旋钮。

高效的LLM推理服务最近的工作提出了通过异构资源[4]和平台[35]、内存和键值缓存管理[9]、[26]以及节点和集群级调度[3]、[30]、[42]、[48]、[50]、[73]、[81]来提高LLM推理效率的方法。虽然这些研究侧重于提高吞吐量或延迟，但我们表明，优化LLM推理的能源效率在性能、能源消耗和开销之间表现出明显的权衡，因此需要一个全面的框架。

七. 结论

我们推出 DynamoLLM，这是第一个用于 LLM 推理集群的能源管理框架。DynamoLLM 利用推理计算属性的异构性和推理工作负载的波动来节省能源。系统在性能保证下运行时，会自动动态地配置集群的能源优化组织（实例数量、模型并行性和 GPU 频率）。DynamoLLM 分别为客户减少了 53%、38% 和 61% 的能源、碳排放和成本。

参考

- [1] “CAISO 碳排放”，<https://www.caiso.com/todays-outlook/emissions>，2024 年。[2] “WattTime”，<https://watttime.org>，2024 年。[3] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani 和 R. Ramjee, “SARATHI: 通过 Piggybacking 进行高效 LLM 推理”使用分块预填充进行解码，”2023 年。[4] K. Alizadeh, I. Mirzadeh, D. Belenko, K. Khataamifard, M. Cho, C. C. D. Mundo, M. Rastegari 和 M. Farajtabar, “法学硕士瞬间：有限内存下的高效大型语言模型推理”，2024 年。[5] J. B. owne, “使用大型语言模型”<https://biomedsciences.unimelb.edu.au/study/dlh/assets/documents/large-language-models-in-education/llms-in-education>，2024。[6] S. Chen, C. Delimitrou 和 J. F. Martínez, “PARTIES: 用于多种交互式服务的 QoS 感知资源分区”，*Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*, 2019 年。[7] S. Chen, A. Jin, C. Delimitrou 和 J. F. Martínez, “ReTail: 选择学习简单性以在云中启用 QoS 感知电源管理”，*Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA '22)*, 2022 年。[8] Cloud Price, “标准 ND96is H100 v5”，<https://cloudprice.net/vm/> 标准 ND96is H100 v5, 2024 年。[9] T. Dao, D. Y. Fu, S. Ermon, A. Rudra 和 C. Ré, “FlashAttention: 具有 IO 意识的快速且内存高效的精确注意”，2022 年。[10] C. Delimitrou 和 C. Kozyrakis, “Amdahl 法则尾部延迟”，*Commun. ACM*, 卷. 61, 没有。2018 年 7 月 8 日。[11] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruehle, L. V. S. Lakshmanan 和 A. Awadallah, “混合大语言模型：成本效益和质量感知的查询路由”，*Proceedings of the International Conference on Learning Representations (ICLR '24)*, 2024 年。[12] A. Faz, S. Kaneda, R. Wang, R. Osi, P. Sharma, F. Chen 和 L. Jiang, “LLM Carbon: 对大型语言模型的端到端碳足迹进行建模”，*arXiv preprint arXiv:2309.14393*, 2024 年。[13] GitHub, “世界上采用最广泛的人工智能开发工具”，<https://github.com/features/copilot>, 2024 年。[14] M. Halpern, B. Boroujerdian, T. Mummert, E. Duesterwald 和 V. Janapa Reddi, “一刀切：通过容差层量化和揭示机器学习云服务 API 中的准确性与延迟权衡”，载于 *Proceedings of the IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS '19)*, 2019 年。[15] M. E. Haque, Y. He, S. Elnikety, T. D. Nguyen, R. Bianchini 和 K. S. McKinley, “利用异质性提高尾部延迟和能源效率”，*Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '17)*, 2017 年。[16] C.-H. Hsu, Q. Deng, J. Mars 和 L. Tang, “SmoothOperator: 减少大型数据中心的电源碎片并提高电源利用率”，*Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18)*, 2018 年。[17] C.-H. Hsu, Y. Zhang, M. A. Laurenzano, D. Meisner, T. Wenisch, J. Mars, L. Tang 和 R. G. Dreslinski, “肾上腺素：通过快速升压精确定位和控制尾部查询”，载于 *Proceedings of the IEEE 21st International Symposium on High Performance Computer Architecture (HPCA '15)*, 2015 年。[18] A. Jahanshahi, M. Rezvani 和 D. Wong, “WattWiser: 多模型多 GPU 推理服务器的功耗和资源高效调度”，*Proceedings of the 14th International Green and Sustainable Computing Conference (IGSC '23)*, 2023 年。[19] Y. Jin, C.-F. Wu, D. 布鲁克斯和 G.-Y. Wei, “S³: 在生成推理过程中提高 GPU 利用率以获得更高的吞吐量”，*Proceedings of the Advances in Neural Information Processing Systems (NeurIPS '23)*, 2023 年。[20] A. K. Kakolyris, D. Masouros, S. Xydis 和 D. Soudris, “用于节能 LLM 推理服务的 SLO 感知 GPU DVFS”，*IEEE Computer Architecture Letters*, 2024 年。[21] H. Kasture, D. B. Bartolini, N. Beckmann 和 D. Sanchez, “Rubik: 延迟关键型系统的快速分析电源管理”，*Proceedings of the 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '15)*, 2015 年。[22] Y. G. Kim 和 C.-J. Wu, “AutoScale: 使用强化学习实现随机边缘推理的能源效率优化”，*Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '20)*, 2020 年。[23] T. Komoda, S. Hayashi, T. Nakada, S. Miwa 和 H. Nakamura, “通过协调 dvfs 和任务映射来限制 cpu-gpu 异构系统的功率上限”，*Proceedings of the IEEE 31st International Conference on Computer Design (ICCD '13)*, 2013。[24] N. Kulkarni, G. Gonzalez-Pumariega, A. Khurana, C. A. Shoemaker, C. Delimitrou 和 D. H. Albonese, “CuttleSys: 可重配置多核交互式服务的数据驱动资源管理”，*Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '20)*, 2020 年。[25] A. G. Kumbhare, R. Azimi, I. Manousakis, A. Bonde, F. Frujeri, N. Mahalingam, P. A. Misra, S. A. Javadi, B. Schroeder, M. Fontoura 和 R. Bianchini, “云平台中基于预测的电力超额订阅”，*Proceedings of the USENIX Annual Technical Conference (USENIX ATC '21)*, 2021 年。[26] W. Kwon, Z. Li, S. Zhuang, Y. Shen, L. Cheng, C. H. Yu, J. E. Gonzalez, H. Zhuang 和 I. Stoica, “使用 PagedAttention 服务的大型语言模型的高效内存管理”，*Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023 年。[27] M. Lammertyn, “60+ ChatGPT 统计数据和你需要了解的事实 2024 年”，<https://blog.invgate.com/chatgpt-statistics>, 2024 年。[28] LC G Consulting, “能源在线：ERCOT 实时价格”，<https://energyonline.com/Data/GenericData.aspx?DataId=4&ERCOT> 实时价格, 2024 年。[29] S. Li, X. Wang, X. Zhang, V. Kontorinis, S. Kodakara, D. Lo 和 P. Ranganathan, “Thunderbolt: 大规模吞吐量优化、服务质量感知功率上限”，载于 *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 2020 年。[30] Z. Li, L. Cheng, Y. Zhong, V. Liu, Y. Shen, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, 和 I. Stoica, “AlpaServe: 深度学习服务的模型并行统计复用”，*Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI '23)*, 2023。[31] D. Lo, L. Cheng, R. Govindaraju, L. A. Barroso 和 C. Kozyrakis, “实现大规模延迟关键型工作负载的能量比例”，*Proceedings of the ACM/IEEE 41st International Symposium on Computer Architecture (ISCA '14)*, 2014。[32] Meta, “Llama2-13B”，<https://huggingface.co/meta-llama/Llama-2-13b>, 2024。[33] Meta, “Llama2-70B”，<https://huggingface.co/meta-llama/Llama-2-70b>, 2024。[34] Meta, “Llama3-70B”，<https://huggingface.co/meta-llama/Meta-Llama-3-70B-Instruct>, 2024 年。

- [35] X. Miao, C. Shi, J. Duan, X. Xi, D. Lin, B. Cui 和 Z. Jia, “SpotServe: 在可抢占实例上提供生成大型语言模型”, *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 2024 年。[36] Microsoft Azure, “ND H100 v5 系列”, <https://learn.microsoft.com/en-us/azure/virtual-machines/nd-h100-v5-series>, 2024 年。[37] A. Mirhosseini 和 T. Wenisch, “ μ Steal: 混合关键性微服务中抢占式工作和资源窃取的理论支持框架”, *Proceedings of the ACM International Conference on Supercomputing (ICS '21)*, 2021 年。[38] Mistral AI, “The Mistral-8x22B 大型语言模型”, <https://huggingface.co/mistralai/Mistral-8x22B-Instruct-v0.1>, 2024。[39] Mistral AI, “Mistral-8x7B 大型语言模型”, <https://huggingface.co/mistralai/Mistral-8x7B-Instruct-v0.1>, 2024。[40] S. M. Nabavinejad, S. Reda 和 M. Ebrahimi, “BatchSizer: DNN 推理的功耗性能权衡”, *Proceedings of the 26th Asia and South Pacific Design Automation Conference (ASP-DAC '21)*, 2021 年。[41] S. M. Nabavinejad, S. Reda 和 M. Ebrahimi, “GPU 加速器上 dnn 推理的协调批处理和 dvfs”, *IEEE Transactions on Parallel and Distributed Systems*, 卷。33, 没有。2022 年 10 月。[42] K. K. W. Ng, H. M. Demoulin 和 V. Liu, “Paella: 采用软件定义 GPU 调度的低延迟模型服务”, 载于 *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 2023 年。[43] R. Nishtala, V. Petrucci, P. Carpenter 和 M. Sjlander, “Twig: 多智能体任务管理共置延迟关键云服务”, *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA '20)*, 2020 年。[44] NVIDIA, “DGX H100: 企业 AI”, <https://www.nvidia.com/en-us/data-center/dgx-h100/>, 2024 年。[45] NVIDIA, “NVLink 和 NVLink 交换机”, <https://www.nvidia.com/en-us/data-center/nvlink/>, 2024 年。[46] NVIDIA, “系统管理接口 SMI”, <https://developer.nvidia.com/system-management-interface>, 2024 年。[47] NVIDIA, “TensorRT-LLM 文档”, <https://nvidia.github.io/TensorRT-LLM/>, 2024 年。[48] H. Oh, K. Kim, J. Kim, S. Kim, J. Lee, D.-s. Chang 和 J. Seo, “Ex-eGPT: LLM 推理的约束感知资源调度”, 载于 *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 2024 年。[49] P. Patel, E. Choukse, C. 张, I. Goiri, B. Warriar, N. Mahalingam 和 R. Bianchini, “描述 LLM 中的电源管理机会 Cloud”, 载于 *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 2024 年。[50] P. Patel, E. Choukse, C. Zhuang, A. Shah, I. Goiri, S. Maleki 和 R. Bianchini, “Splitwise: 使用相分裂的高效生成式 LLM 推理”, 载于 *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*, 2024 年。[51] T. Patel 和 D. Tiwari, “CLITE: 仓库规模计算机的多个延迟关键作业的高效和 QoS 感知共定位”, 载于 *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA '20)*, 2020 年。[52] C. Peng, X. Yang, A. Chen, K. Smith, N. PourNejatian, A. Costa, C. Martin, M. Flores, Y. Zhang, T. Magoc, G. Lipori, M. Duane, N. Ospina, M. Ahmed, W. Hogan, E. Shenkman, Y. Guo, J. Bian 和 Y. Wu, “医学研究和医疗保健的生成大语言模型的研究”, *npj Digital Medicine*, 2023 年。[53] Python PuLP, “使用 PuLP 进行优化”, <https://coin-or.github.io/pulp/>, 2024 年。[54] Python SciPy, “SciPy Library”, <https://scipy.org/>, 2024 年。[55] H. Qiu, W. Mao, A. Patke, S. Cui, S. Jha, C. Wang, H. Franke, Z. T. Kalboczyk, T. Basar 和 R. K. Iyer, “基于代理模型的序列长度预测的高效交互式 LLM 服务”, 载于 *The 5th International Workshop on Cloud Intelligence / AIOps at ASPLOS 2024*, 2024。[56] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, “语言模型是无监督的多任务学习者”, *OpenAI blog*, 卷。1, 没有。8, p. 2019 年 9 月 9 日。[57] F. Romero, Q. Li, N. J. Yadwadkar 和 C. Kozyrakis, “INFaaS: 自动化无模型推理服务”, 载于 *Proceedings of the USENIX Annual Technical Conference (USENIX ATC '21)*, 2021 年。[58] S. Samsi, D. Zhu, J. McDonald, B. Li, A. Michaleas, M. Jones, W. Bergeron, J. Kepner, D. Tiwari 和 V. Gadepally, “从单词到瓦特: 对大型语言模型推理的能源成本进行基准测试”, 参见 *2023 IEEE High Performance Extreme Computing Conference (HPEC)*。IE EE, 2023 年, 第 1-9 页。[59] T. L. Scao, A. Fan, C. Akiki, E. Pavlick 和 S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé *et al.*, “BLOOM: 176b 参数开放获取多语言语言模型”, *arXiv preprint arXiv:2211.05100*, 2022 年。[60] J. Stojkovic, E. Choukse, C. 张, I. Goiri 和 J. Torrelas, “迈向绿色大语言模型: 能源效率带到法学硕士推理的前沿”, *arXiv e-prints*, 第 17 页。arXiv:2403.20306, 2024 年 3 月。[61] J. Stojkovic, N. Iliakopoulou, T. Xu, H. Franke 和 J. Torrelas, “EcoFaaS: 重新思考无服务器环境设计以提高能源效率”, *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*, 2024 年。[62] J. Stojkovic, P. Misra, I. Goiri, S. Whitlock, E. Choukse, M. Das, C. Bansal, J. Lee, Z. Sun, H. Qiu, R. Zimmermann, S. Samal, B. Warriar, A. Raniwala 和 R. Bianchini, “SmartOClock: 云中的工作负载和风险感知超频”, 载于 *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA '24)*, 2024 年。[63] K. Talamadupula, “LLM 推理性能监控指南”, <https://syml.ai/developers/blog/a-guide-to-llm-inference-performance-monitoring/>, 2024。[64] T. Tambe, C. Hooper, L. Pentecost, T. Jia, E.-Y. Yang, M. Donato, V. Sanh, P. Whatmough, A. M. Rush, D. Brooks 和 G.-Y. Wei, “EdgeBERT: 用于延迟感知多任务 NLP 推理的句子级能量优化”, 载于 *Proceedings of the 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21)*, 2021 年。[65] J. Z. Tang, Y. Wang, Q. Wang 和 X. Chu, “GPU DVFS 对深度学习的能量和性能的影响: 实证研究”, 载于 *Proceedings of the Tenth ACM International Conference on Future Energy Systems (e-Energy '19)*, 2019 年。[66] 技术创新研究所 (TII), “Falcon-180B”, 2024 年。[67] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bosale *et al.*, “Llama 2: 开放基础和微调聊天模型” *arXiv preprint arXiv:2307.09288*, 2023。[68] T. Varshney, “构建用于数据分析的 llm 驱动的数据代理”, <https://developer.nvidia.com/blog/build-an-llm-powered-data-agent-for-data-analysis/>, 2024。[69] C. Wan, M. Santraja, E. Rogers, H. Hoffmann, M. Maire 和 S. Lu, “警告: 能源和及时性的准确学习”, 载于 *Proceedings of the USENIX Annual Technical Conference (USENIX ATC '20)*, 2020 年。[70] F. Wang, W. Zhang, S. Lai, M. hao 和 Z. Wang, “机器学习训练工作负载的动态 GPU 能源优化”, *IEEE Transactions on Parallel and Distributed Systems*, 2022 年。[71] W. Xiao, S. Ren, Y. Li, Y. Zhuang, P. Hou, Z. Li, Y. Feng, W. Lin 和 Y. Jia, “AntMan: 用于深度学习的 GPU 集群动态扩展”, 载于 *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20)*, 2020 年。[72] J. You, J.-W. Chung 和 M. Chowdhury, “Zeus: 理解和优化 DNN 训练的 GPU 能耗”, *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*, 2023 年。[73] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim 和 B.-G. Chun, “Orca: 基于 Transformer 的生成模型的分布式服务系统”, *Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*, 2022 年。[74] J. Yu, J. Kim 和 E. Seo, “了解你的敌人以节省云能源: 机器学习服务的能源性能表征”, *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023 年。[75] C. Zhang, A. G. Kumbhare, I. Mano usakis, D. Zhang, P. A. Misra, R. Assis, K. Woolcock, N. Mahalingam, B. Warriar, D. Gauthier, L. Kunnath, S. Solomon, O. Morales, M. Fontoura 和 R. Bianchini, “Flex: 零预留功率的高可用性数据中心”, *Proceedings of the 48th Annual International Symposium on Computer Architecture (ISCA '21)*, 2021 年。[76] Y. 张, W. Hua, Z. Zhou, G. E. Suh 和 C. Delimitrou, “Sinan: 基于 ML 和 QoS 感知的云微服务资源管理”, *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*, 2021 年。

[77] Y.Zhang、Q.Wang、Z.Lin、P.Xu 和 B.Wang, “通过具有性能保证的应用程序透明频率扩展策略提高 GPU 能效”, 载于 *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*, 2024 年。[78] Y.Z.Zhao、D.W.Wu 和 J.Wang, “ALISA: 通过稀疏感知 KV 缓存加速大型语言模型推理”, 载于 *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*, 2024 年。[79] Z. Cheng、X. Ren、F. Xu、Y. Luo、X. Jiang 和 Y. You, “响应长度感知和序列调度: LLM 授权的 LLM 推理管道”, *Proceedings of the Advances in Neural Information Processing Systems (NeurIPS '23)*, 2023 年。

[80] L. Zhou、L. N. Bhuyan 和 K. K. Ramakrishnan, “Gemini: 学习管理延迟关键型搜索引擎的 CPU 功率”, 载于 *Proceedings of the 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '20)*, 2020 年。[81] Z. Zhou、X. Wei、J. 张和 G. Sun, “PetS: 参数高效变压器服务的统一框架”, 载于 *Proceedings of the USENIX Annual Technical Conference (USENIX ATC '22)*, 2022 年。[82] P. Zou、A. Li、K. Barker 和 R. Ge, “GPU 加速系统上迭代工作负载的指标导向动态电源管理”, *Proceedings of the 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID '20)*, 2020 年。